

ADR-01: Arquitectura Base del Sistema

Multiverse Rumble — Juego de Peleas 1v1 en Pixel Art

Autor	Gael
Fecha	15/05/2026
Estado	Propuesto

1. Contexto

Multiverse Rumble es un videojuego de peleas 1v1 con estética pixel art, desarrollado como aplicación web con ASP.NET Core MVC. El proyecto nace del debate eterno entre fans: ¿quién ganaría entre Goku y Superman? ¿Naruto o Luffy? ¿Scooby-Doo o Coraje el perro cobarde?

La aplicación permite que dos jugadores en la misma computadora elijan cada uno un personaje de distintos universos (anime, caricaturas, cómics, videojuegos) y se enfrenten en tiempo real. A futuro se contempla la modalidad multijugador en línea.

Condiciones y restricciones que influyeron en la decisión:

- El proyecto es individual y se desarrolla durante un cuatrimestre.
- El lenguaje y framework principal requerido por la materia es C# con ASP.NET Core MVC.
- El desarrollador tiene conocimiento previo en .NET, Razor Views y Bootstrap.
- La pelea en tiempo real requiere lógica del lado del cliente que MVC no cubre de forma nativa.

2. Decisión

Se elige ASP.NET Core MVC con C# como arquitectura principal del sistema, combinado con JavaScript Canvas para el motor del combate en el navegador.

MVC gestiona todo lo administrativo: registro de personajes, habilidades, escenarios, historial de combates y perfiles de jugadores. JavaScript Canvas se encarga del renderizado de sprites pixel art y la lógica de pelea en tiempo real.

Tecnología	Rol en el sistema
C# / ASP.NET Core	Backend y lógica de negocio
MVC (Modelo-Vista-Controlador)	Patrón de arquitectura principal
Razor Views (.cshtml)	Generación de vistas del servidor
JavaScript + Canvas API	Motor de combate en tiempo real
Bootstrap 5	Diseño responsivo e interfaz
Git + GitHub	Control de versiones

3. Alternativas Consideradas

Alternativa	Por qué la descarté
Unity + WebGL	Genera archivos muy pesados para el navegador y requiere aprender un motor completamente distinto al stack de la materia.
Django (Python) + Pygame	No cumple con el requisito de la materia de usar .NET MVC. Además, Pygame no está diseñado para aplicaciones web.
Scratch	Es una plataforma visual para niños. No maneja modelos, controladores ni base de datos. No es válido para un proyecto universitario de arquitectura.
Blazor WebAssembly	Aunque es .NET, no es el patrón MVC solicitado. Además tiene una curva de aprendizaje mayor y menos documentación disponible.

4. Consecuencias

✓ Lo que gano:

Consecuencia técnica: MVC separa claramente la lógica del negocio (modelos), la interfaz (vistas) y el flujo de datos (controladores). Esto hace que sea más fácil agregar nuevos personajes, escenarios o modos de juego sin romper lo que ya funciona.

Consecuencia sobre el proceso: Al usar tecnologías conocidas (.NET, Bootstrap, JavaScript), el tiempo de desarrollo se reduce y puedo avanzar por módulos: primero personajes, luego escenarios, finalmente el combate.

⚠ Lo que sacrifico o asumo:

Limitación técnica: ASP.NET Core MVC no está diseñado para lógica de videojuego en tiempo real. El combate debe manejarse completamente en JavaScript en el navegador, lo que divide la responsabilidad entre el servidor y el cliente y puede volverse complejo de mantener si el juego crece.

Deuda técnica: La modalidad multijugador en línea no está contemplada en esta arquitectura. Para implementarla en el futuro se necesitaría agregar SignalR (WebSockets) al proyecto, lo que implicaría refactorizar parte del motor de combate.

5. Diagrama del Sistema

Multiverse Rumble — Diagrama de Arquitectura MVC

